

Mémoria dinámica

Parte 2

Estructuras de Datos, 2026

Dpto. Lenguajes y Ciencias de la
Computación.

Universidad de Málaga



Modularidad en C

- La modularidad se logra mediante la separación del código en archivos de encabezado (*.h) y archivos de implementación (*.c).
- Los archivos de encabezado contienen las declaraciones de las funciones y estructuras que se utilizarán en otros archivos
- Los archivos de implementación contienen las definiciones de esas funciones y estructuras.

Modularidad en C: ventajas

- Facilita la reutilización del código
- Mejora la legibilidad y el mantenimiento del código
- Ayuda a evitar la duplicación de código y a mantener una estructura organizada del proyecto.

Modularidad en C: librerías

```
// Incluye el archivo de encabezado de la biblioteca estándar de C
#include <stdio.h>

// Incluye el archivo de encabezado local, comillas dobles
#include "miarchivo.h"

int main(void) {
    // Código del programa
    return 0;
}
```

Ejercicio: Punto

Modularidad en C: cabeceras

```
#ifndef PUNTO_H // Evita la inclusión múltiple del archivo de encabezado
#define PUNTO_H

struct Punto{
    int x;
    int y;
}; // Declaración de una estructura

struct Punto suma(struct Punto a, struct Punto b); // Declaración de una función

#endif
```

Modularidad en C: implementación

```
#include "punto.h" // Incluye el archivo de encabezado local

struct Punto suma(struct Punto a, struct Punto b) {

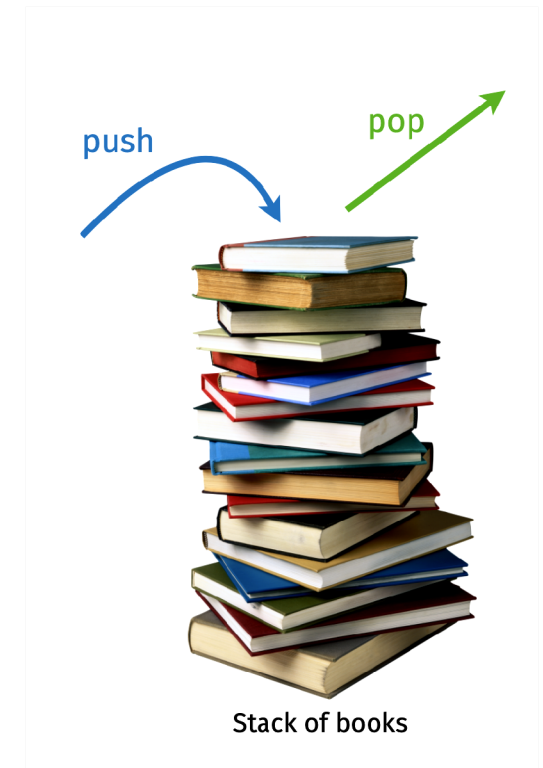
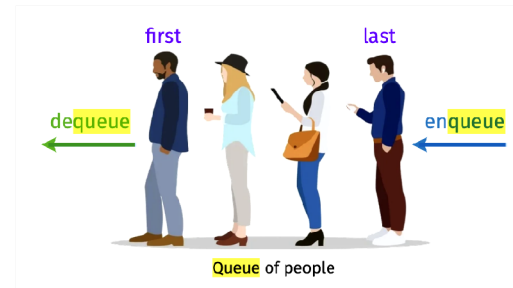
    a.x += b.x;
    a.y += b.y;
    return a;
}
```

Compilar código con librerías

```
gcc -g ./Principal.c ./punto.c -Wall -Wextra -Wpedantic
```

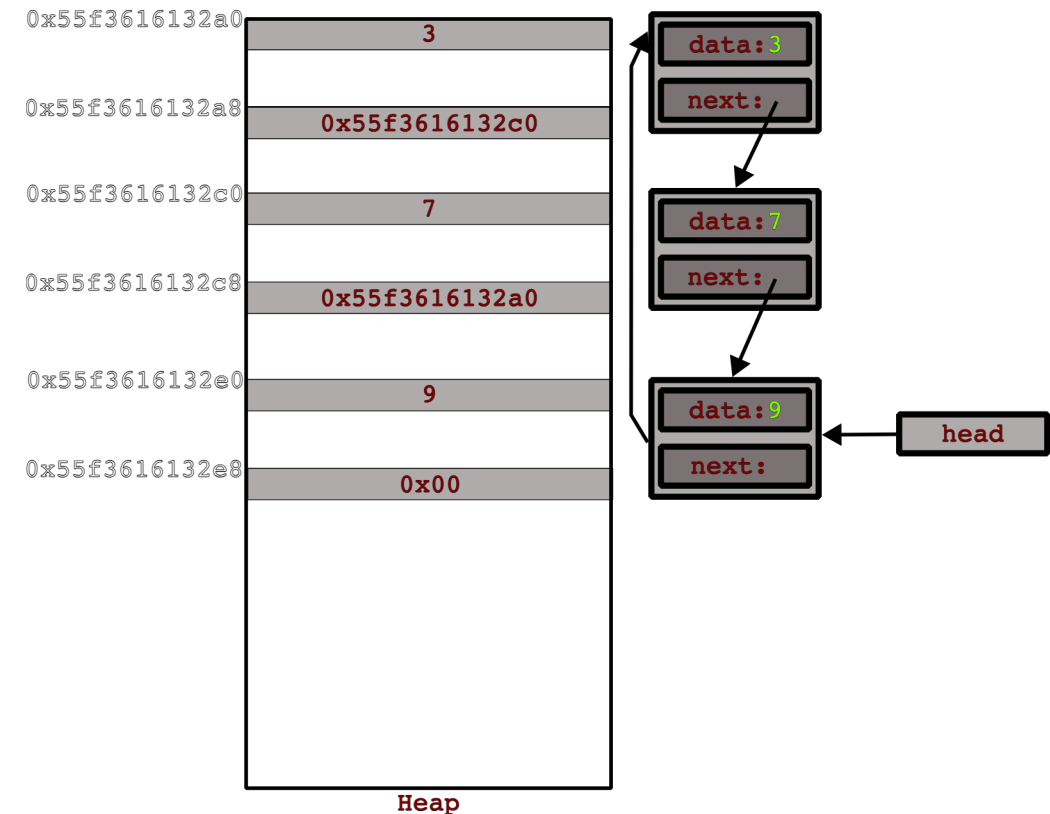
Ejercicios para practicar la modularidad y las estructuras enlazadas

- Cola (Estructura FIFO *First-In-First-Out*)
- Pila (lista/array) (Estructura LIFO *Last-In-First-Out*)



Ejercicio Cola

- Es una estructura *First-In-First-Out*, por lo que se insertará en la cola y se extraerá de la cabeza de la lista.
- Vamos a implementarlo con una **lista circular**.
 - *head* apunta al último elemento insertado (cola de la lista).
 - *head->next* es el primer elemento insertado (cabeza de la lista).

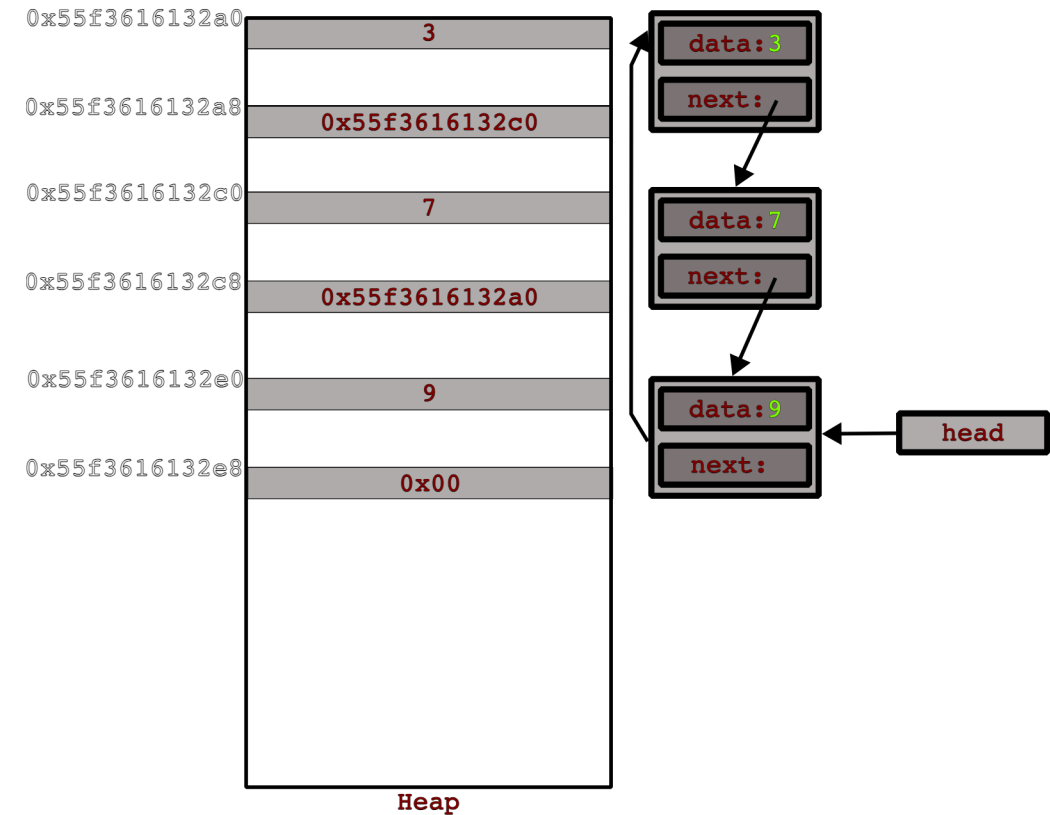


Cabecera

```
struct Person
{
    char name[30];
    int age;
};

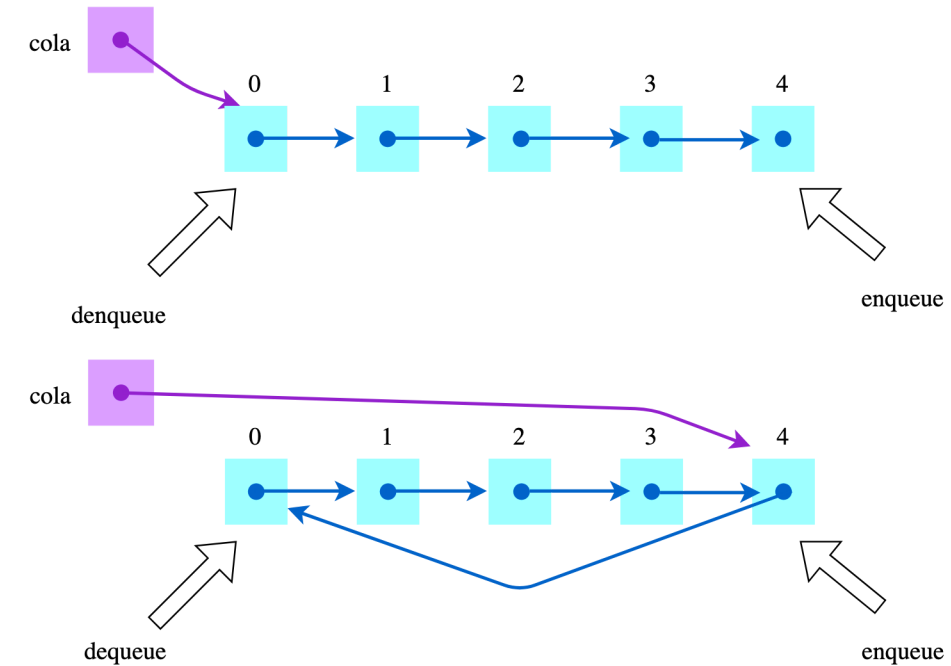
typedef struct Node *Queue;
struct Node
{
    struct Person * person;
    struct Node *next;
};

struct Person * createPerson(char *name, int age);
void createQueue(struct Node **ptrqueue);
bool isEmpty(struct Node * queue);
int size(struct Node * queue);
void enqueue(struct Node **ptrqueue, struct Person * person);
void dequeue(struct Node **ptrqueue);
struct Person * first(struct Node * queue);
void clear(struct Node **ptrqueue);
void display(struct Node * queue);
```



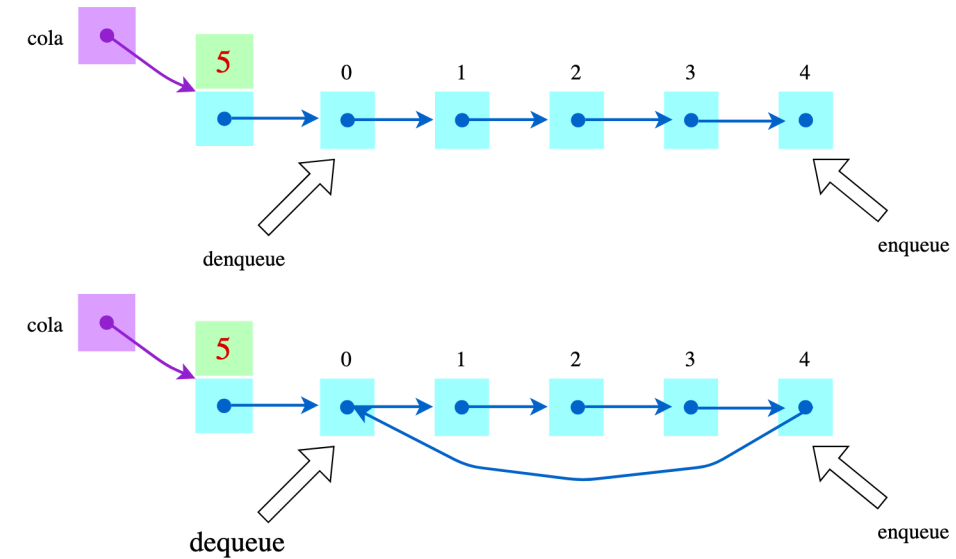
Cola: implementaciones 1/2

Operación	Lista Enlazada	Lista Circular
enqueue	$O(n)$	$O(1)$
dequeue	$O(1)$	$O(1)$
size	$O(n)$	$O(n)$



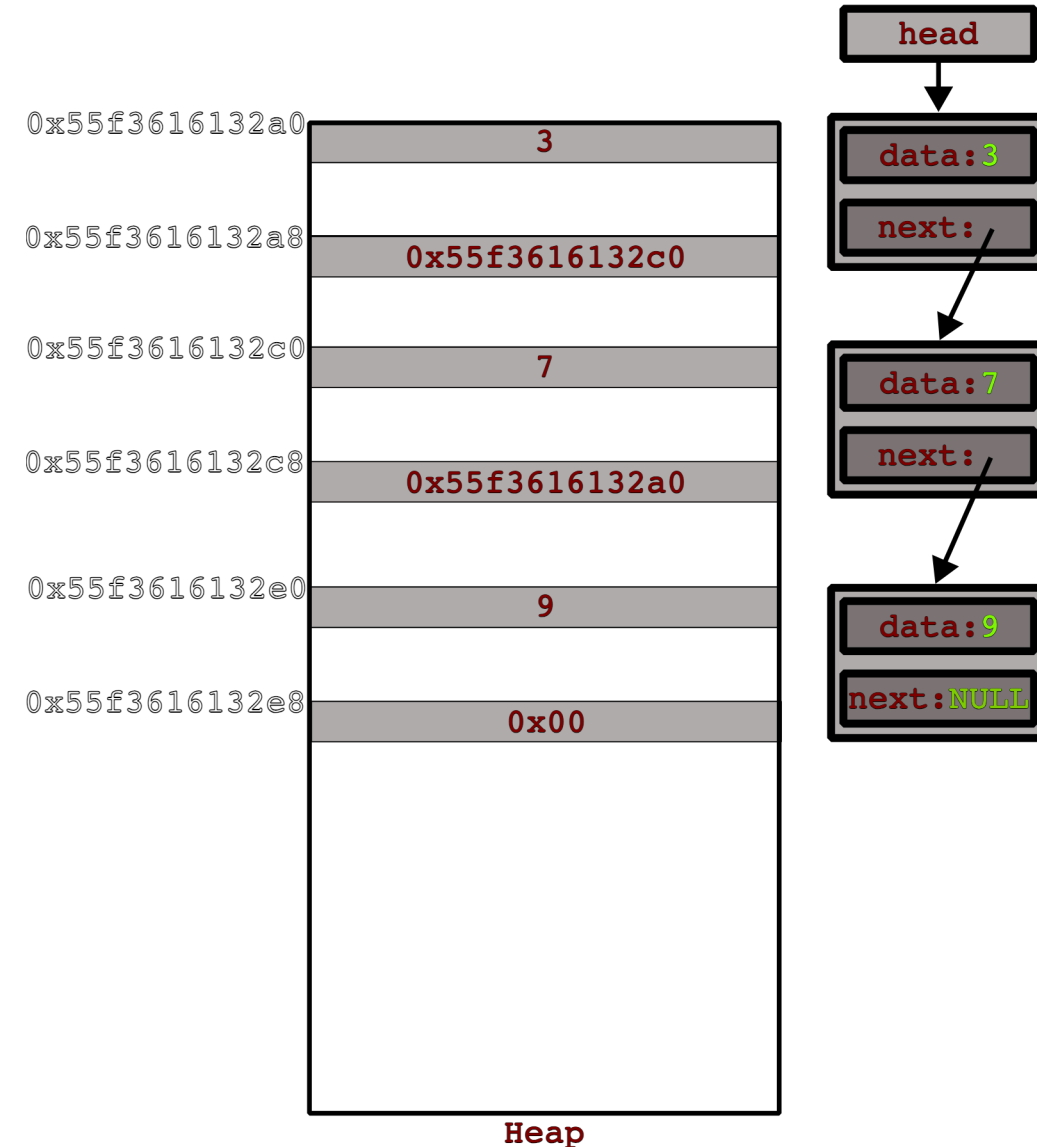
Cola: implementaciones 2/2

Método	Lista Enlazada	Lista Circular
enqueue	$O(n)$	$O(1)$
dequeue	$O(1)$	$O(1)$
size	$O(1)$	$O(1)$



Ejercicio Pila con lista enlazada

- Es una estructura *Last-In-First-Out*, por lo que se insertará en la cabeza y se extraerá de la cabeza de la lista.
- Vamos a implementarlo con una **lista enlazada**.
 - *head* apunta al último elemento insertado (cabeza de la lista).



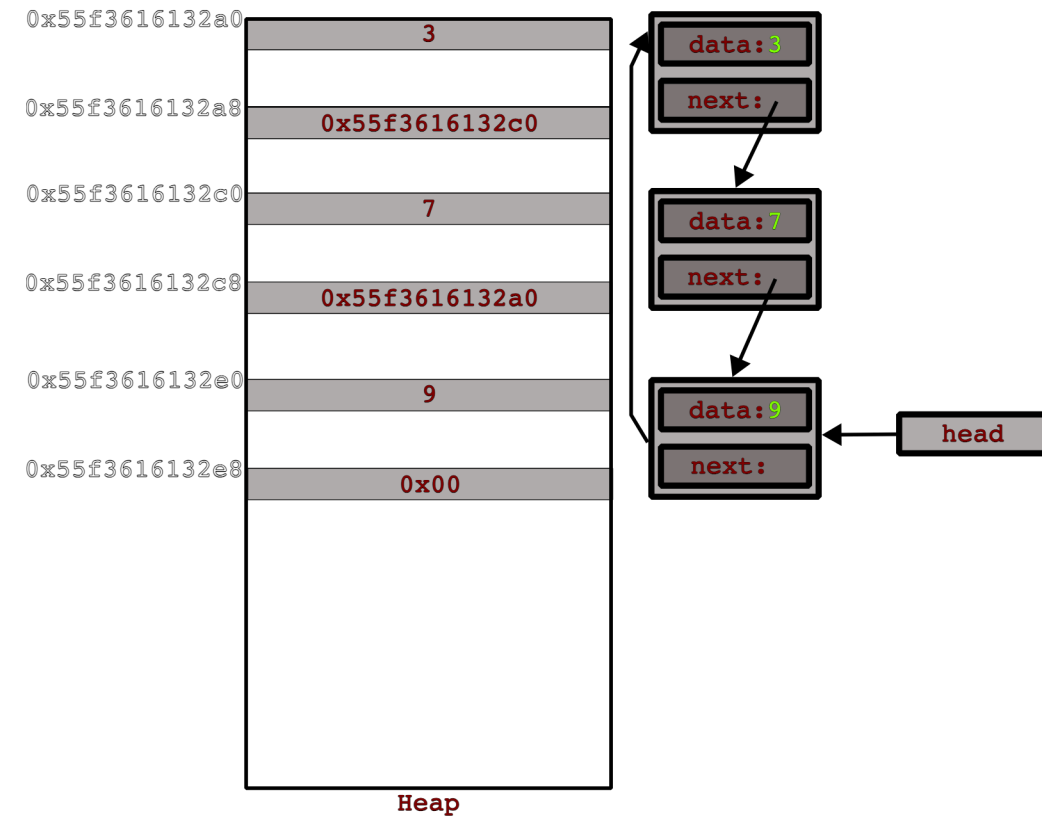
Cabecera

```
enum Tipo {PLASTICO, METAL, MADERA};

struct Node{
    int color;
    float capacidad;
    enum Tipo material;
    struct Node *next;
};

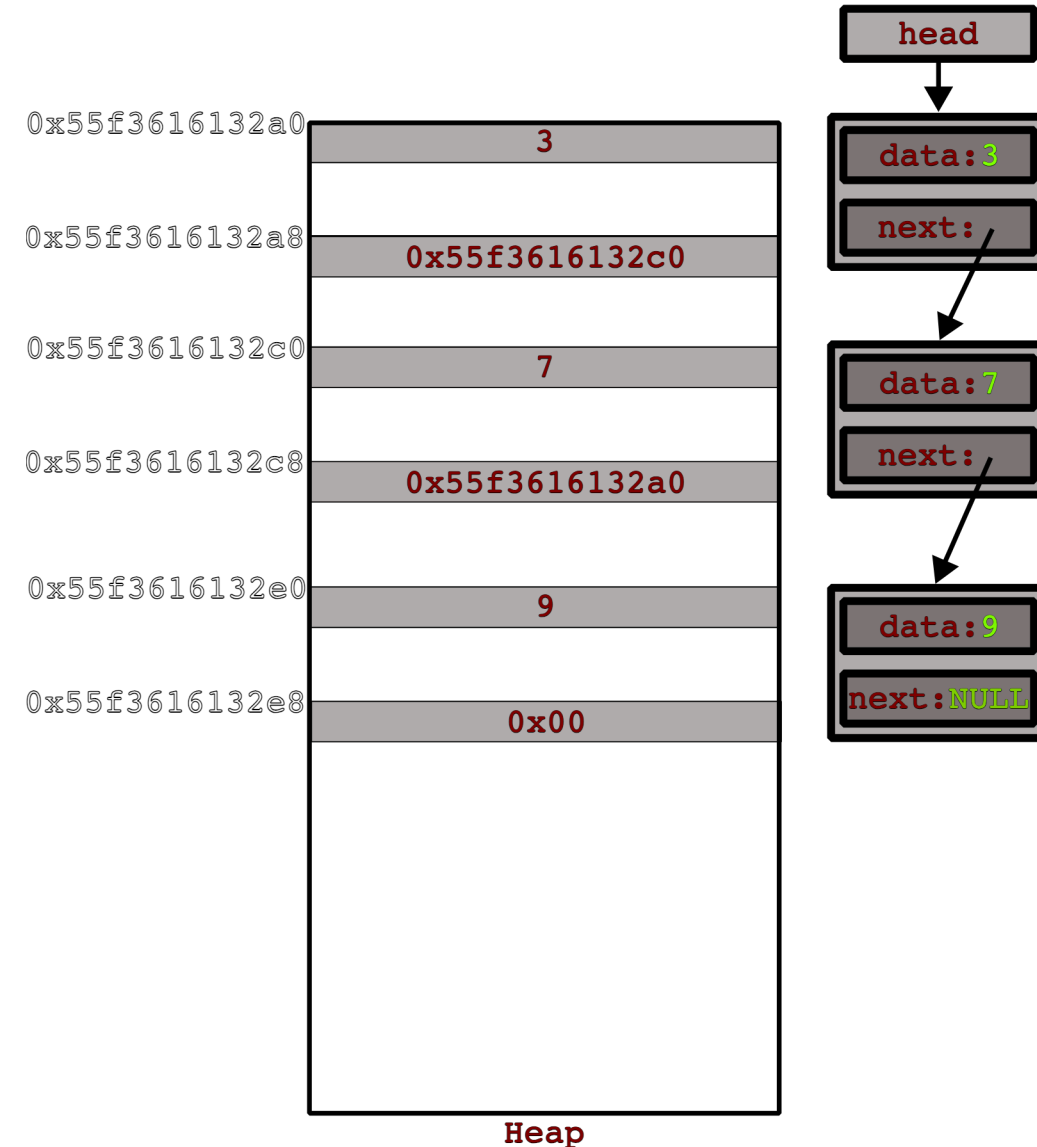
struct Stack_ll{
    Node *top;
    unsigned size;
};

void stack_ll_init(struct Stack_ll **ptrStack);
void stack_ll_push(struct Stack_ll *ptrStack, struct Node element);
struct Node stack_ll_top(struct Stack_ll * ptrStack);
bool stack_ll_pop(struct Stack_ll * ptrStack);
bool stack_ll_isEmpty(struct Stack_ll * ptrStack);
int stack_ll_size(struct Stack_ll * ptrStack);
void stack_ll_clear(struct Stack_ll * ptrStack);
void stack_ll_show(struct Stack_ll * ptrStack);
```



Ejercicio Pila con array

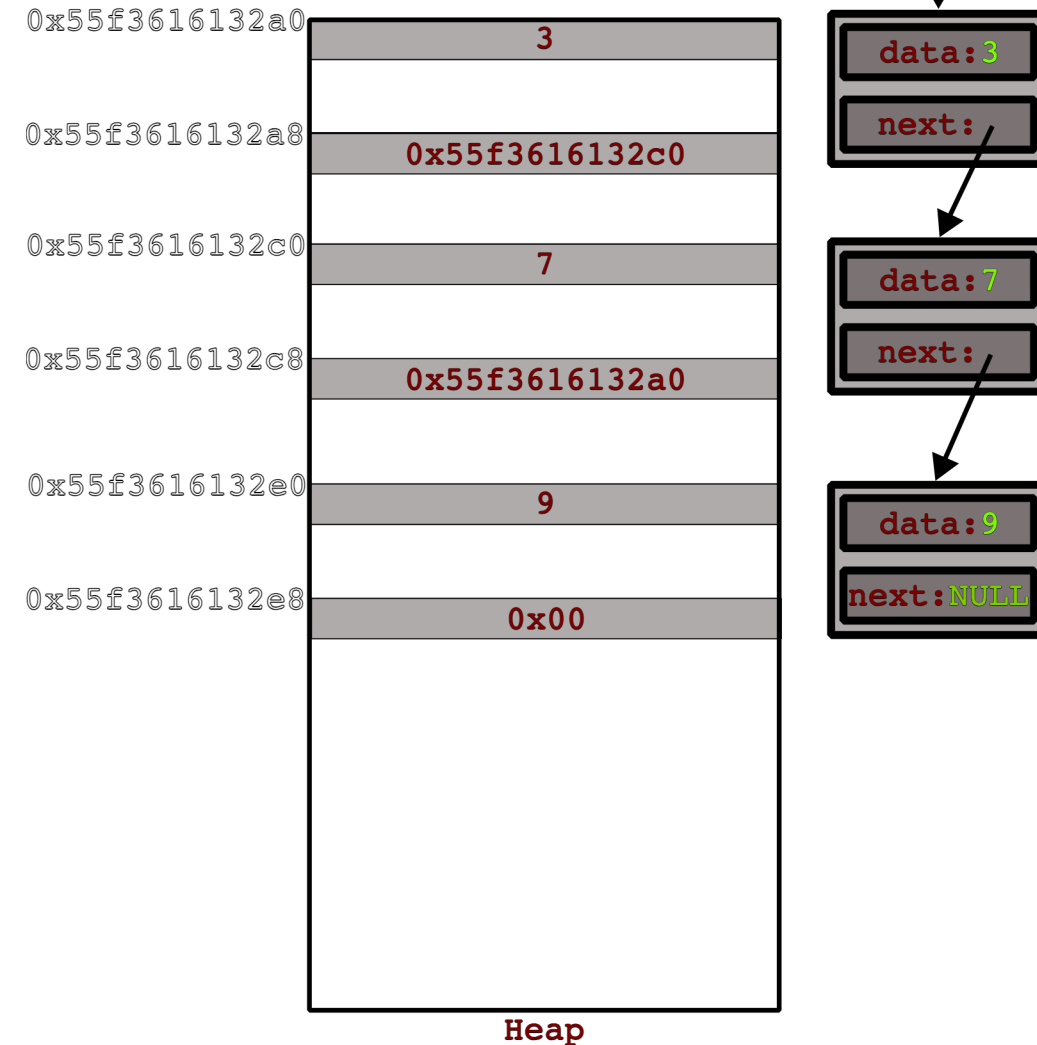
- Es una estructura *Last-In-First-Out*, por lo que se insertará en la cabeza y se extraerá de la cabeza de la lista.
- Vamos a implementarlo con **array**. Usaremos **realloc** para **duplicar su capacidad cuando esté lleno**.



Cabecera

```
enum Tipo{PLASTICO, METAL, MADERA};
//typedef struct Node Node; //Mismo que lo de abajo
struct Node{
    int color;
    float capacidad;
    enum Tipo material;
};
struct Stack_a{
    struct Node *top;
    unsigned size;
    unsigned capacity;
};

void stack_a_init(struct Stack_a * ptrStack_a, unsigned capacity);
void stack_a_push(struct Stack_a * ptrStack_a, struct Node element);
struct Node stack_a_top(struct Stack_a * ptrStack_a);
bool stack_a_pop(struct Stack_a * ptrStack_a);
bool stack_a_is_empty(struct Stack_a * ptrStack_a);
int stack_a_size(struct Stack_a * ptrStack_a);
void stack_a_clear(struct Stack_a * ptrStack_a);
void stack_a_show(struct Stack_a * ptrStack_a);
```



Pila: implementaciones

Método	Lista Enlazada	Array
push	$O(1)$	$O(1)$ $O(n)$ resize
pop	$O(1)$	$O(1)$
top	$O(1)$	$O(1)$
size	$O(1)$	$O(1)$

